

টেকনিক্যাল আর্কিটেকচার ডায়াগ্রাম ডকুমেন্ট

University Club Platform

ফাইল-টু-ফাইল কানেকশন আর্কিটেকচার · ডাটাবেজ ডিজাইন · ডিজাইন প্যাটার্ন ডায়াগ্রাম

Backend File Map

Frontend ↔ Backend Map

ER Diagram

Pattern Diagrams

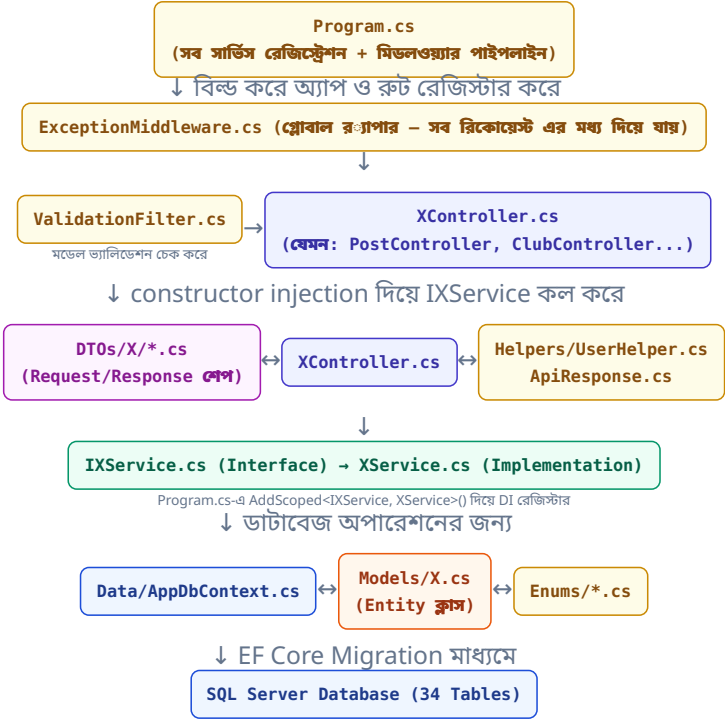
📖 এই ডকুমেন্টে দেখানো হয়েছে — ঠিক কোন ফাইল কোন ফাইলের সাথে কানেক্টেড, কীভাবে ডাটা প্রবাহিত হয়, এবং ব্যাকএন্ড-ফ্রন্টএন্ড কীভাবে যুক্ত।
🔗 আগের ডকুমেন্ট (প্রজেক্ট ওভারভিউ)-এর সাথে একসাথে পড়ার জন্য তৈরি — এটি সম্পূরক (companion) ডকুমেন্ট।

১	ব্যাকএন্ড ফাইল কানেকশন আর্কিটেকচার (সামগ্রিক)	পৃষ্ঠা ৩
২	নির্দিষ্ট মডিউল উদাহরণ: Post তৈরি — ফাইল-টু-ফাইল ফ্লো	পৃষ্ঠা ৪
৩	Controller → Service → Model সম্পূর্ণ ম্যাপিং টেবিল	পৃষ্ঠা ৫
৪	SignalR Hub ফাইল কানেকশন	পৃষ্ঠা ৭
৫	ডাটাবেজ ER ডায়াগ্রাম (Entity Relationship)	পৃষ্ঠা ৮
৬	ডিজাইন প্যাটার্ন — ভিজুয়াল ডায়াগ্রাম	পৃষ্ঠা ১০
৭	ফ্রন্টএন্ড ফাইল কানেকশন আর্কিটেকচার	পৃষ্ঠা ১৩
৮	ফ্রন্টএন্ড ↔ ব্যাকএন্ড সম্পূর্ণ কানেকশন ম্যাপ	পৃষ্ঠা ১৪
৯	উপসংহার	পৃষ্ঠা ১৬

১ ব্যাকএন্ড ফাইল কানেকশন আর্কিটেকচার (সামগ্রিক)

নিচের ডায়াগ্রামে দেখানো হয়েছে ব্যাকএন্ডের প্রতিটি স্তরের ফাইল একে অপরের সাথে ঠিক কীভাবে কানেক্টেড। তীরচিহ্নের দিক নির্দেশ করে "কে কাকে কল করে"।

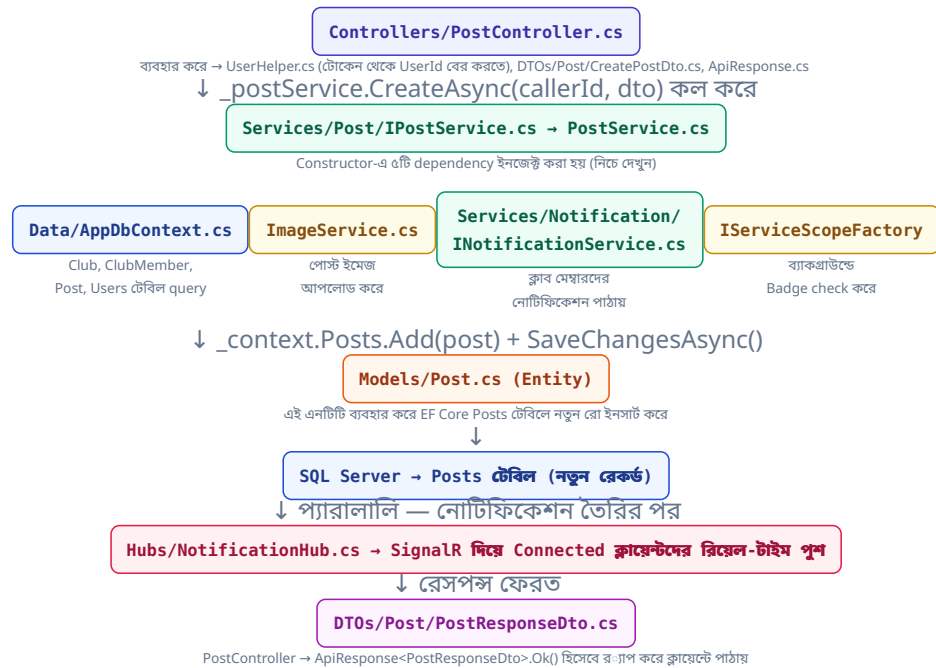
Controller ফাইল Service ফাইল Model / Entity ফাইল DTO ফাইল Helper / Filter / Middleware DbContext / Database



📌 **মূল নিয়ম:** Controller কখনো সরাসরি `AppDbContext` বা `Model` স্পর্শ করে না — সবসময় Service Interface-এর মাধ্যমে যায়। এটি Controller ও Database-এর মধ্যে একটি স্পষ্ট বিভাজন (Decoupling) তৈরি করে, যাতে ভবিষ্যতে Database পরিবর্তন করলেও Controller-এর কোড পরিবর্তনের দরকার না হয়।

২ নির্দিষ্ট মডিউল উদাহরণ: "Post তৈরি" — প্রকৃত ফাইল-টু-ফাইল ফ্লো

এখানে প্রকৃত কোড থেকে যাচাই করা "Post তৈরি" ফিচারের সম্পূর্ণ ফাইল কানেকশন দেখানো হলো — এটি বুঝতে সাহায্য করবে বাস্তবে কোন ফাইল কোন ফাইল ইমপোর্ট/কল করে।



✦ কোড থেকে নিশ্চিত করা হয়েছে: PostService কনস্ট্রাক্টরে ইনজেক্ট করা হয় — AppDbContext context , ImageService imageService , INotificationService notificationService , IServiceScopeFactory scopeFactory , ILogger<PostService> logger । অর্থাৎ একটি Service প্রয়োজনে একাধিক অন্য Service ব্যবহার করতে পারে — এটি Service-to-Service Composition-এর উদাহরণ।

৩

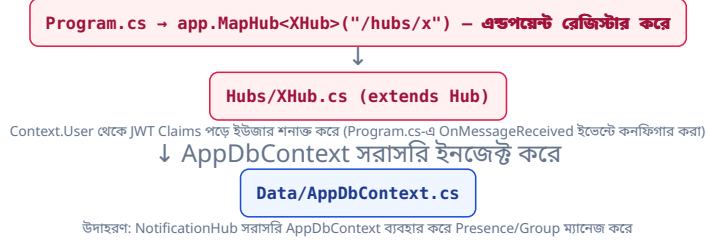
Controller → Service → Model সম্পূর্ণ ম্যাপিং টেবিল

নিচের টেবিলে প্রতিটি Controller ফাইল কোন Service ব্যবহার করে এবং সেই Service কোন Model/টেবিল নিয়ে কাজ করে তা দেখানো হলো — এটি সম্পূর্ণ ব্যাকএন্ডের "ফাইল কানেকশন ম্যাপ"।

Controller ফাইল	ব্যবহৃত Service (Interface → Impl)	মূল Model/টেবিল	অতিরিক্ত কানেকশন
AuthController.cs	IAuthService → AuthService.cs	User.cs	JwtHelper.cs, IEmailService
ClubController.cs	IClubService → ClubService.cs	Club.cs, ClubMember.cs	ClubPermissionHelper.cs
ClubPrivacyController.cs	IClubPrivacyService → ClubPrivacyService.cs	Club.cs (Visibility)	—
PostController.cs	IPostService → PostService.cs	Post.cs, SavedPost.cs, PostReport.cs	ImageService.cs, INotificationService
CommentController.cs	ICommentService → CommentService.cs	Comment.cs	INotificationService
ReactionController.cs	IReactionService → ReactionService.cs	Reaction.cs, CommentReaction.cs	INotificationService
FeedController.cs	IFeedService → FeedService.cs	Post.cs, Follow.cs, ClubMember.cs	—
EventController.cs	IEventService → EventService.cs	Event.cs, EventAttendance.cs	INotificationService
LiveEventController.cs	ILiveEventService → LiveEventService.cs	LiveParticipant.cs, LiveChatMessage.cs	Hubs/LiveEventHub.cs
MessageController.cs	IMessageService → MessageService.cs	Message.cs	Hubs/ChatHub.cs
GroupController.cs	IGroupService → GroupService.cs	Group.cs, GroupMember.cs, GroupMessage.cs	Hubs/GroupHub.cs
NotificationController.cs	INotificationService → NotificationService.cs	Notification.cs	Hubs/NotificationHub.cs
FollowController.cs	IFollowService → FollowService.cs	Follow.cs, BlockedUser.cs	INotificationService
PollController.cs	IPollService → PollService.cs	Poll.cs, PollOption.cs, PollVote.cs	—
StoryController.cs	IStoryService → StoryService.cs	Story.cs, StoryView.cs	ImageService.cs
RecruitmentController.cs	IRecruitmentService → RecruitmentService.cs	ClubApplication.cs, ClubInvite.cs	INotificationService, IEmailService
BadgeController.cs	IBadgeService → BadgeService.cs	Badge.cs, UserBadge.cs	—
LeaderboardController.cs	ILeaderboardService → LeaderboardService.cs	Post, Event, Badge, Follow (Aggregate Query)	—
SearchController.cs	ISearchService → SearchService.cs	SearchHistory.cs (+multi-entity query)	—
RecommendationController.cs	IRecommendationService → RecommendationService.cs	ClubRecommendationDismissal.cs	Services/AI/IGeminiService
DashboardController.cs	IDashboardService → DashboardService.cs	Multi-entity Aggregate	—
FileController.cs	IFileService → FileService.cs	FileResource.cs	wwwroot/uploads/
VoiceMessageController.cs	IVoiceMessageService → VoiceMessageService.cs	Message.cs / GroupMessage.cs (Voice Type)	wwwroot/uploads/voice/
PresenceController.cs	IPresenceService → PresenceService.cs	User.cs (IsOnline, LastSeenAt)	Hubs/AppHub.cs
UserController.cs	IUserService → UserService.cs	User.cs, ProfileView.cs	—

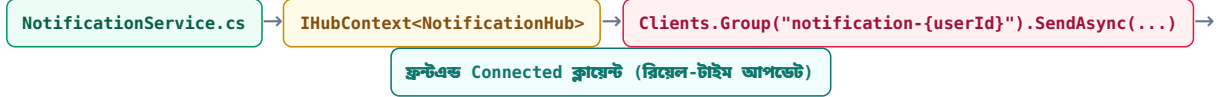
প্যাটার্ন লক্ষ্য করুন: প্রতিটি সারিতে একই কাঠামো — একটি Controller একটি নির্দিষ্ট Service Interface-এর সাথে ১:১ সম্পর্কে যুক্ত, এবং সেই Service একটি বা একাধিক Model নিয়ে কাজ করে। যেসব ফিচারে রিয়েল-টাইম প্রয়োজন (Message, Group, Notification, Live Event), সেগুলোর Service সংশ্লিষ্ট SignalR Hub-ও ব্যবহার করে।

SignalR Hub গুলো সাধারণ Controller-Service ফ্লো থেকে আলাদা পথে কাজ করে — এগুলো সরাসরি WebSocket কানেকশনের মাধ্যমে ক্লায়েন্টের সাথে দ্বিমুখী যোগাযোগ করে।



Hub থেকে Service-এ বিপরীতমুখী কানেকশন (Push)

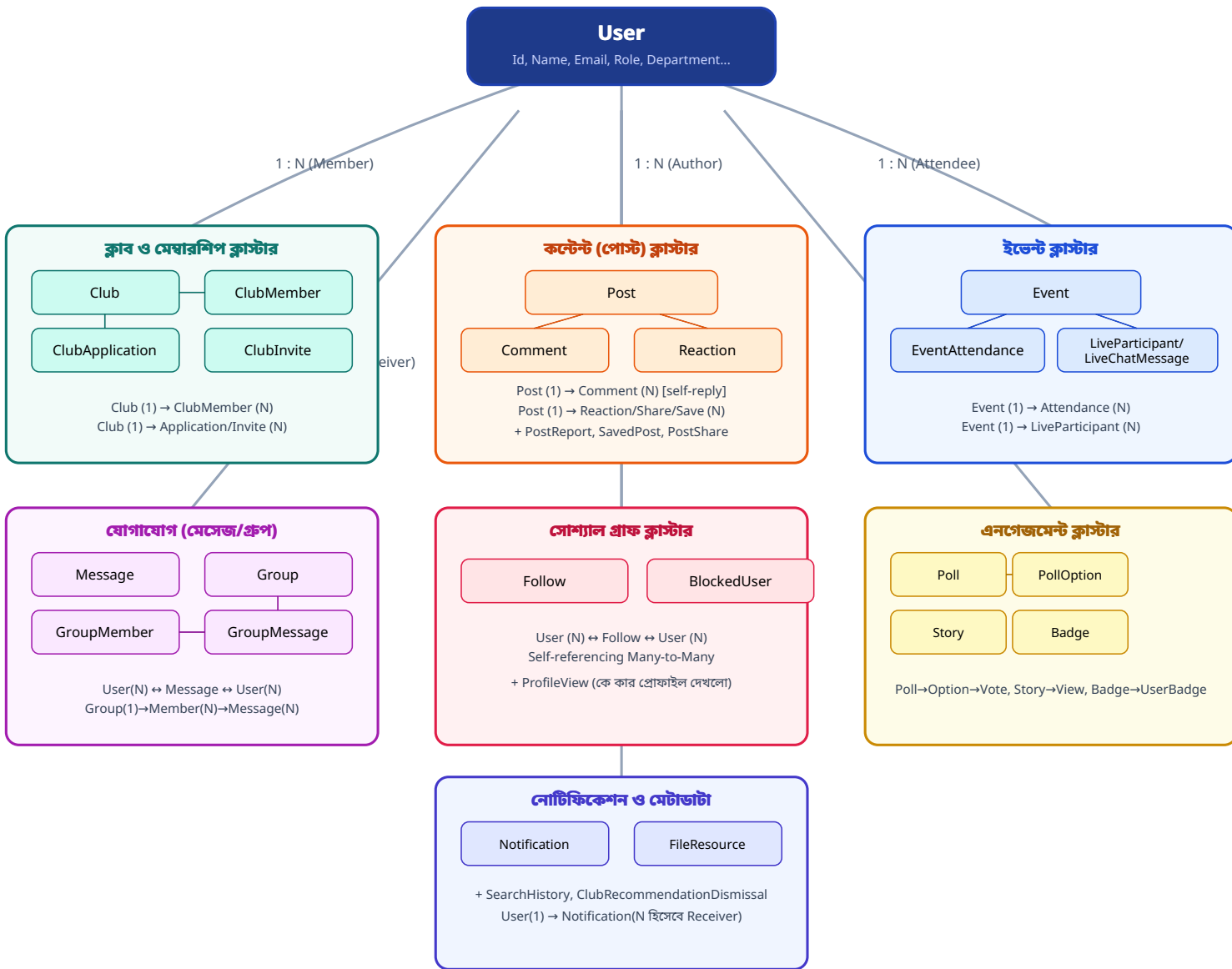
যখন কোনো Service (যেমন NotificationService) কোনো ইভেন্ট তৈরি করে, তখন সেটি `IHubContext<NotificationHub>` ইনজেক্ট করে সংশ্লিষ্ট Hub-কে কল করে ক্লায়েন্টে push পাঠায়:



Hub ফাইল	এন্ডপয়েন্ট	কোন Service/Controller থেকে ট্রিগার হয়
Hubs/ChatHub.cs	/hubs/chat	MessageService.cs (নতুন মেসেজ পাঠানোর সময়)
Hubs/GroupHub.cs	/hubs/group	GroupService.cs (গ্রুপ মেসেজ পাঠানোর সময়)
Hubs/NotificationHub.cs	/hubs/notification	NotificationService.cs (যেকোনো নোটিফিকেশন ইভেন্টে — Post, Comment, Follow, Event...)
Hubs/LiveEventHub.cs	/hubs/live	LiveEventService.cs (লাইভ চ্যাট, পার্টিসিপেন্ট জয়েন/লিভ, মডারেশন)
Hubs/AppHub.cs	(কমন)	PresenceService.cs (Online/Offline Status)

৫ ডাটাবেজ ER ডায়াগ্রাম (Entity Relationship)

নিচে **User**-কে কেন্দ্র করে মূল ৭টি ডোমেইন ক্লাস্টারের সম্পর্ক দেখানো হলো — প্রতিটি ক্লাস্টারের ভেতরে সংশ্লিষ্ট এনটিটিগুলোর নিজস্ব সম্পর্ক আছে।

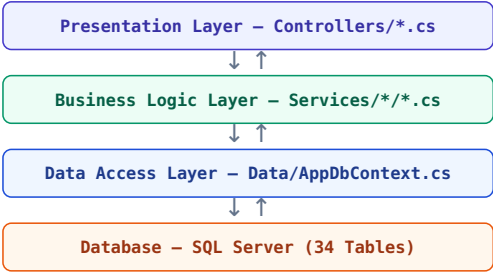


মূল ইনডেক্সিং ও কনস্ট্রেন্ট নীতি (সব ক্লাস্টার জুড়ে প্রযোজ্য)

- ✓ প্রতিটি "জয়েন টেবিল"-এ (Many-to-Many সংযোগ) কম্পোজিট Unique Index — যেমন (UserId, PostId), (UserId, ClubId)
- ✓ ঘন ঘন query হওয়া কলামে Index — CreatedAt, ReceiverId, EventDate ইত্যাদি
- ✓ Pending স্ট্যাটাসের জন্য Filtered Unique Index — ClubApplication ও ClubInvite-এ
- ✓ ডিফল্ট DeleteBehavior.Restrict — ভুলবশত Cascade Delete প্রতিরোধ করতে
- ✓ প্যারেন্ট-নির্ভর চাইল্ড ডাটায় (Post→Comment, Club→Member) সুনির্দিষ্ট Cascade Delete
- ✓ Soft-delete Query Filter — User.cs ও FileResource.cs-এ (IsDeleted = false)

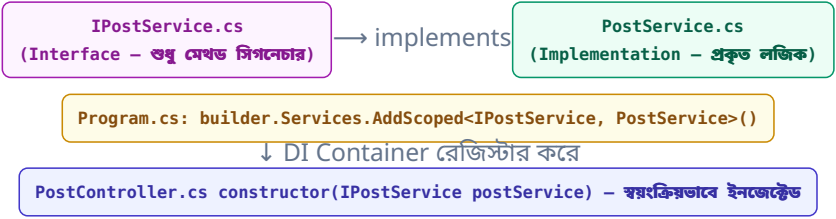
মোট: ৩৪টি এনটিটি, ৭টি ডোমেইন ক্লাস্টার, ২৭টি EF Core Migration, ১টি কেন্দ্রীয় User এনটিটি

৬.১ Layered Architecture Pattern



প্রতিটি স্তর শুধু তার ঠিক নিচের স্তরের সাথে কথা বলে — Controller কখনো সরাসরি Database স্পর্শ করে না, সবসময় Service ও DbContext-এর মধ্য দিয়ে যায়।

৬.২ Dependency Injection (DI) Pattern

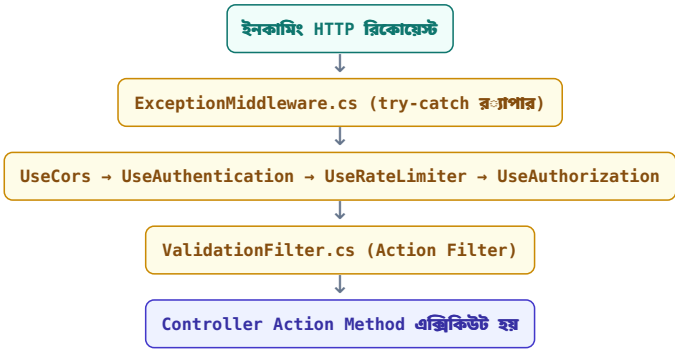


৬.৩ DTO Pattern (Data Transfer Object)



DTO ব্যবহার করার ফলে Entity-র Password, RefreshToken-এর মতো সংবেদনশীল কলাম কখনো ক্লায়েন্টে যায় না।

৬.৪ Middleware / Pipeline Pattern



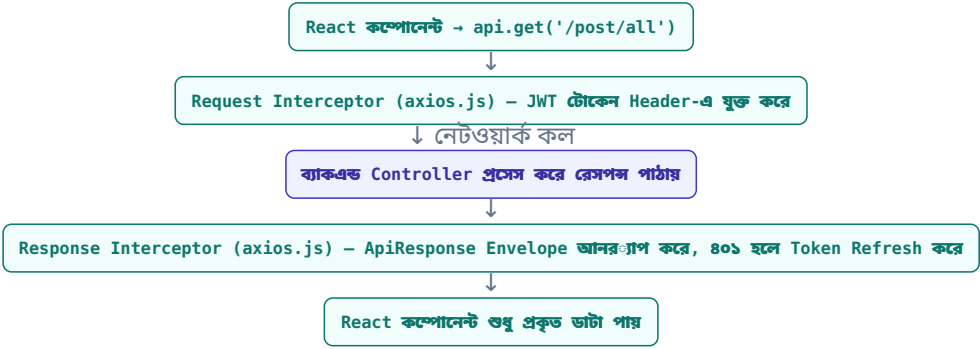
প্রতিটি রিকোয়েস্ট এই "পাইপলাইন" এর প্রতিটি ধাপ ক্রমানুসারে অতিক্রম করে — যেকোনো ধাপে ব্যর্থ হলে (যেমন Auth Fail) পরের ধাপে যায় না।

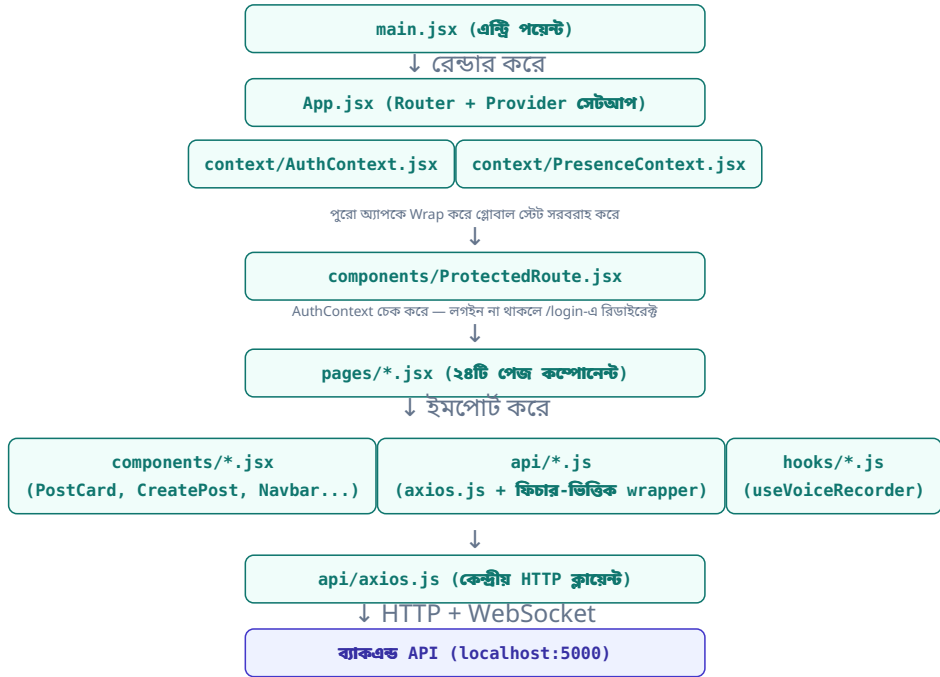
৬.৫ Observer Pattern (SignalR Push)



যতগুলো ক্লায়েন্ট নির্দিষ্ট Group-এ Subscribe করা আছে (যেমন `notification-{userId}`), তারা সবাই একসাথে রিয়েল-টাইম আপডেট পায় — এটি ক্লাসিক Publish-Subscribe/Observer প্যাটার্ন।

৬.৬ Interceptor Pattern (Axios — Frontend)





নিচের টেবিলে দেখানো হলো — কোন ফ্রন্টএন্ড ফাইল কোন পদ্ধতিতে (সরাসরি axios.js নাকি ডেডিকেটেড wrapper ফাইল) ব্যাকএন্ডের কোন Controller-এর সাথে কানেক্টেড।

ফ্রন্টএন্ড পেজ/কম্পোনেন্ট	API সংযোগ পদ্ধতি	ব্যাকএন্ড Controller
pages/Feed.jsx	সরাসরি <code>api.get("/post/all")</code> (axios.js)	PostController.cs
components/CreatePost.jsx	সরাসরি <code>api.post("/post/create")</code>	PostController.cs
components/PostCard.jsx	সরাসরি axios.js ইনস্ট্যান্স ইমপোর্ট	PostController.cs, ReactionController.cs, CommentController.cs
components/Poll/* .jsx	api/poll.js → <code>pollApi</code> wrapper অবজেক্ট	PollController.cs (/api/polls/...)
components/Story/* .jsx	api/story.js → <code>storyApi</code> wrapper	StoryController.cs
components/Recruitment/* .jsx	api/recruitment.js → <code>recruitmentApi</code> wrapper	RecruitmentController.cs (/api/recruitment/...)
components/ClubPrivacy/* .jsx	api/clubPrivacy.js wrapper	ClubPrivacyController.cs
pages/Leaderboard.jsx	api/leaderboard.js wrapper	LeaderboardController.cs
pages/Search.jsx	api/search.js wrapper	SearchController.cs
pages/Recommendations.jsx	api/recommendation.js wrapper	RecommendationController.cs → Services/AI/GeminiService.cs
pages/LiveEvent.jsx	api/liveEvent.js (REST) + api/liveEventHub.js (WebSocket)	LiveEventController.cs + Hubs/LiveEventHub.cs
context/PresenceContext.jsx	api/presence.js wrapper	PresenceController.cs + Hubs/AppHub.cs
components/VoiceRecorderBar.jsx	api/voiceMessage.js wrapper	VoiceMessageController.cs
pages/Notifications.jsx	axios.js + api/notificationHub.js (WebSocket)	NotificationController.cs + Hubs/NotificationHub.cs
pages/Messages.jsx	axios.js + SignalR ChatHub কানেকশন	MessageController.cs + Hubs/ChatHub.cs
pages/Groups.jsx	axios.js + SignalR GroupHub কানেকশন	GroupController.cs + Hubs/GroupHub.cs
pages/Clubs.jsx, ClubDetails.jsx	সরাসরি axios.js	ClubController.cs
pages/Events.jsx	সরাসরি axios.js	EventController.cs
pages/Dashboard.jsx	সরাসরি axios.js	DashboardController.cs
pages/Login.jsx, Register.tsx	সরাসরি axios.js → AuthContext.jsx আপডেট	AuthController.cs
pages/Files.jsx	সরাসরি axios.js (multipart/form-data)	FileController.cs

৭. লক্ষণীয় প্যাটার্ন: প্রজেক্টে দুই ধরনের API কানেকশন স্টাইল ব্যবহৃত হয়েছে — (১) সহজ CRUD ফিচারে (Post, Club, Event) সরাসরি `api.get/post()` কল করা হয়, এবং (২) জটিল/বড় ফিচারে (Poll, Story, Recruitment, Search, Leaderboard) একটি ডেডিকেটেড `xxxApi` wrapper অবজেক্ট তৈরি করা হয়েছে, যা `unwrap()` হেল্পার ফাংশন দিয়ে এরর হ্যান্ডলিং কেন্দ্রীভূত করে — এটি একটি হালকা **Facade Pattern**-এর উদাহরণ।

এই ডকুমেন্টে দেখানো হয়েছে University Club Platform প্রজেক্টের প্রতিটি স্তরের ফাইল কীভাবে একে অপরের সাথে প্রকৃতপক্ষে কানেক্টেড — কোডের প্রকৃত `using` স্টেটমেন্ট, Constructor Injection এবং API কল পর্যালোচনা করে এই ডায়াগ্রামগুলো তৈরি করা হয়েছে, যাতে এটি বাস্তব কোডবেসের একটি সঠিক প্রতিচ্ছবি হয়।

এই ডকুমেন্ট থেকে যা শেখা যায়

- ব্যাকএন্ড:** Controller → Service Interface → Service Implementation → AppDbContext → Model → Database — এই সরলরৈখিক ফ্লো সব ফিচারে সামঞ্জস্যপূর্ণভাবে অনুসরণ করা হয়েছে।
- ডাটাবেজ:** User এনটিটিকে কেন্দ্র করে ৭টি স্বতন্ত্র ডোমেইন ক্লাস্টার (Club, Content, Event, Messaging, Social, Engagement, Notification) — প্রতিটির নিজস্ব সুসংগঠিত সম্পর্ক ও কনস্ট্রেন্ট রয়েছে।
- ডিজাইন প্যাটার্ন:** Layered Architecture, DI, DTO, Middleware Pipeline, Observer (SignalR), এবং Interceptor প্যাটার্ন — এই ৬টি প্যাটার্ন একসাথে কাজ করে একটি নির্ভরযোগ্য, রক্ষণাবেক্ষণযোগ্য সিস্টেম তৈরি করেছে।
- ফ্রন্টএন্ড-ব্যাকএন্ড সংযোগ:** axios.js একক কেন্দ্রীয় গেটওয়ে হিসেবে কাজ করে, যেখানে REST কল ও SignalR WebSocket কানেকশন উভয়ই ব্যাকএন্ডের সাথে সমন্বিতভাবে যোগাযোগ করে।